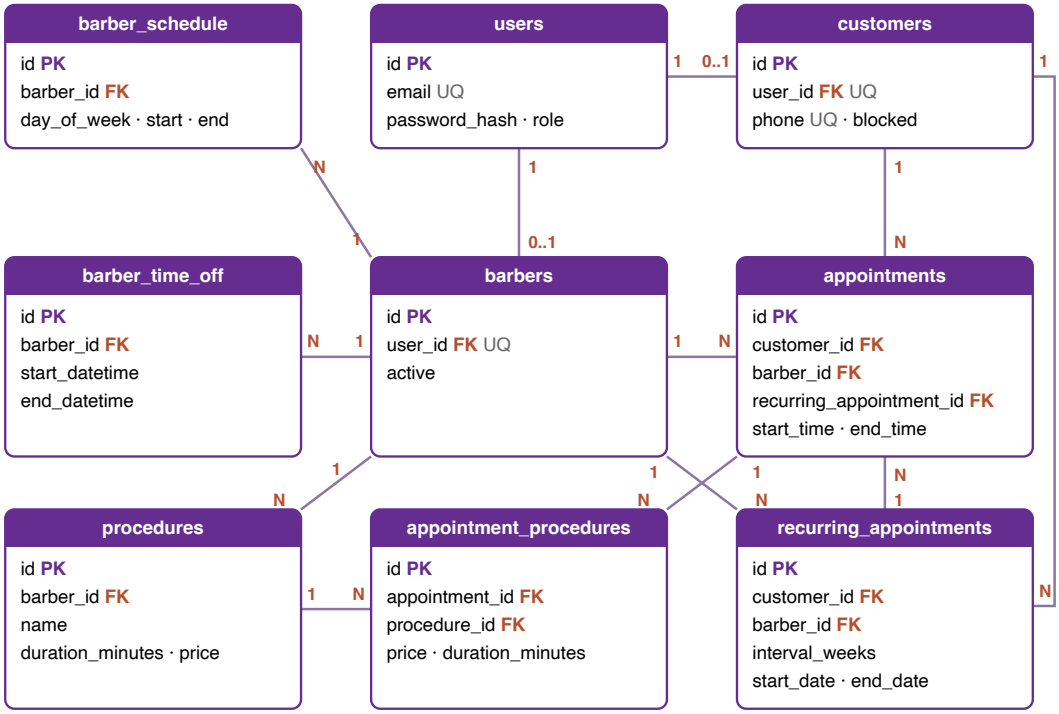
Structura bazei de date

MySQL 8 · 9 tabele · programări la frizerie · Rareș

Diagrama



PK cheie primară, `BIGINT` auto-increment · **FK** cheie străină · **UQ** valoare unică · **NN** obligatoriu (`NOT NULL`)

Tabelele

users

Contul de logare: client, frizer sau administrator (patronul).

coloană	tip	reguli
id	BIGINT	PK
email	VARCHAR(150)	NN, UQ
password_hash	VARCHAR(255)	NN
role	VARCHAR(20)	NN · CUSTOMER, BARBER, ADMIN
created_at	DATETIME	NN

customers

Profilul unui cont cu rolul `CUSTOMER`.

coloană	tip	reguli
id	BIGINT	PK
user_id	BIGINT	NN, UQ, FK → users.id
first_name	VARCHAR(100)	NN
last_name	VARCHAR(100)	NN
phone	VARCHAR(20)	NN, UQ
blocked	BOOLEAN	NN, implicit false
created_at	DATETIME	NN

barbers

Profilul unui cont cu rolul `BARBER`; îl creează administratorul.

coloană	tip	reguli
id	BIGINT	PK
user_id	BIGINT	NN, UQ, FK → users.id
first_name	VARCHAR(100)	NN
last_name	VARCHAR(100)	NN
phone	VARCHAR(20)	NN
active	BOOLEAN	NN, implicit true
created_at	DATETIME	NN

procedures

Ce face un frizer, cu prețul și durata lui: „Tuns” la Mihai, 60 lei, 30 min.

coloană	tip	reguli
id	BIGINT	PK
barber_id	BIGINT	NN, FK → barbers.id
name	VARCHAR(100)	NN
description	VARCHAR(500)	
duration_minutes	INT	NN, > 0
price	DECIMAL(10,2)	NN, ≥ 0
active	BOOLEAN	NN, implicit true

> UQ pe `(barber_id, name)`: doi frizeri pot avea amândoi „Tuns”, fiecare cu prețul lui.

barber_schedule

Programul de lucru pe săptămână: un rând pe zi lucrată.

coloană	tip	reguli
id	BIGINT	PK
barber_id	BIGINT	NN, FK → barbers.id
day_of_week	TINYINT	NN, 1–7 (1 = luni, ca DayOfWeek)
start_time	TIME	NN
end_time	TIME	NN, > start_time

> UQ pe perechea (barber_id, day_of_week). Zi fără rând = zi liberă.

barber_time_off

Concediu, zi liberă, pauză: frizerul nu primește programări.

coloană	tip	reguli
id	BIGINT	PK
barber_id	BIGINT	NN, FK → barbers.id
start_datetime	DATETIME	NN
end_datetime	DATETIME	NN, > start_datetime
reason	VARCHAR(200)	

appointments

O programare a unui client la un frizer.

coloană	tip	reguli
id	BIGINT	PK
customer_id	BIGINT	NN, FK → customers.id
barber_id	BIGINT	NN, FK → barbers.id
recurring_appointment_id	BIGINT	FK → recurring_appointments.id; gol la programările simple
start_time	DATETIME	NN
end_time	DATETIME	NN, = start + suma duratelor
status	VARCHAR(30)	NN · BOOKED, CONFIRMED, CANCELED, COMPLETED
notes	VARCHAR(500)	
created_at	DATETIME	NN

> Două programări ale aceluiași frizer, care nu sunt CANCELED, nu se suprapun. Un UNIQUE nu poate spune asta: verificarea e în service. Anularea schimbă statusul, rândul rămâne.

appointment_procedures

Ce proceduri are o programare.

coloană	tip	reguli
id	BIGINT	PK
appointment_id	BIGINT	NN, FK → appointments.id
procedure_id	BIGINT	NN, FK → procedures.id
price	DECIMAL(10,2)	NN, ≥ 0
duration_minutes	INT	NN, > 0

> UQ pe (appointment_id, procedure_id). Toate procedurile sunt ale frizerului programării. Prețul și durata se copiază la programare: o scumpire nu schimbă programările vechi.

recurring_appointments

„Din 2 în 2 săptămâni, marți la 10:00, până în decembrie.”

coloană	tip	reguli
id	BIGINT	PK
customer_id	BIGINT	NN, FK → customers.id
barber_id	BIGINT	NN, FK → barbers.id
interval_weeks	INT	NN, > 0
start_date	DATE	NN
end_date	DATE	NN, ≥ start_date
start_time	TIME	NN
active	BOOLEAN	NN, implicit true

> Ziua săptămânii = ziua lui start_date. Fiecare programare generată are recurring_appointment_id.

Față de modelele tale din 1b2d672

- Tabel nou users: emailul se mută aici din customers și barbers; customers și barbers primesc user_id.
- recurring_appointments primește barber_id: fără el nu știi la cine generezi programările.
- procedures primește barber_id: fiecare frizer are procedurile lui, cu prețurile lui.
- UQ nou pe (barber_id, name) la proceduri și pe (barber_id, day_of_week) la program.